

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from collections import Counter
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix, ConfusionMatrixDisplay

# Mount Google Drive

from google.colab import drive
drive.mount('/content/drive')

# Load Dataset

file_path = "/content/drive/My Drive/iris.csv"
df = pd.read_csv(file_path)

# Drop Id column safely

if "Id" in df.columns:
    df = df.drop(columns=["Id"])

# KNN Implementation

def euclidean_distance(x1, x2):
    return np.sqrt(np.sum((x1 - x2) ** 2))

class KNN:
    def __init__(self, k=3):
        self.k = k

    def fit(self, X, y):
        self.X_train = np.array(X)
        self.y_train = np.array(y)

    def predict(self, X):
        return [self._predict(x) for x in X]

    def _predict(self, x):
        distances = [euclidean_distance(x, x_train) for x_train in self.X_train]
        k_indices = np.argsort(distances)[:self.k]
        k_labels = [self.y_train[i] for i in k_indices]
        return Counter(k_labels).most_common(1)[0][0]

# Prepare Data

X = df.iloc[:, :-1].values
y = df.iloc[:, -1].values

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Train Model

model = KNN(k=3)
model.fit(X_train, y_train)

predictions = model.predict(X_test)

# Results

print("Predicted Classes:", set(predictions))

for i, pred in enumerate(predictions):
    print(f"Sample {i+1}: {pred}")

```

```
print(f"\nAccuracy: {accuracy_score(y_test, predictions):.2f}")

# Visualization

fig, axes = plt.subplots(1, 2, figsize=(12, 5))

colors = ["#4CAF50", "#2196F3", "#FF5722"]
classes = sorted(set(y))

# Scatter plot
ax = axes[0]
for cls, col in zip(classes, colors):
    mask = y == cls
    ax.scatter(X[mask, 2], X[mask, 3],
               color=col, label=cls,
               s=60, edgecolors="k", linewidths=0.5)

ax.set_xlabel("Petal Length")
ax.set_ylabel("Petal Width")
ax.set_title("Scatter Plot")
ax.legend()
ax.grid(alpha=0.3)

# Confusion Matrix
ax = axes[1]
cm = confusion_matrix(y_test, predictions)
ConfusionMatrixDisplay(cm, display_labels=sorted(set(y_test))).plot(
    ax=ax, colorbar=False, cmap="Blues"
)
ax.set_title("Confusion Matrix")

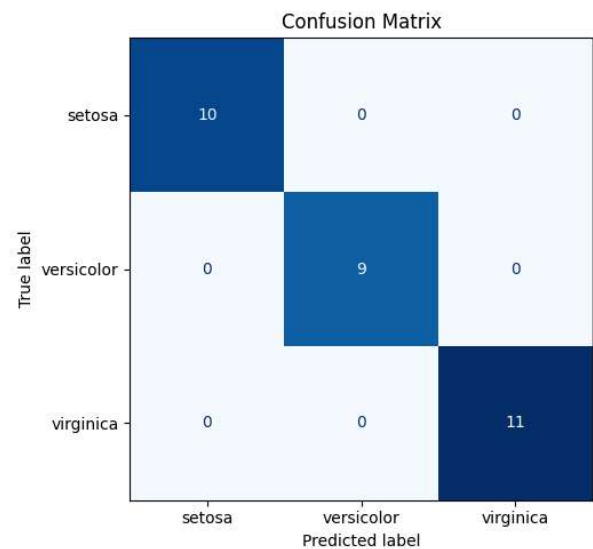
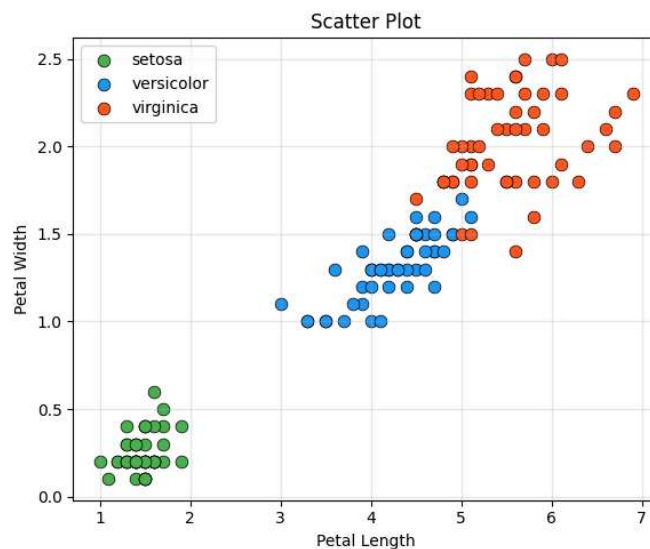
plt.tight_layout()
plt.show()
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True)

Predicted Classes: {'setosa', 'versicolor', 'virginica'}

Sample 1: versicolor
 Sample 2: setosa
 Sample 3: virginica
 Sample 4: versicolor
 Sample 5: versicolor
 Sample 6: setosa
 Sample 7: versicolor
 Sample 8: virginica
 Sample 9: versicolor
 Sample 10: versicolor
 Sample 11: virginica
 Sample 12: setosa
 Sample 13: setosa
 Sample 14: setosa
 Sample 15: setosa
 Sample 16: versicolor
 Sample 17: virginica
 Sample 18: versicolor
 Sample 19: versicolor
 Sample 20: virginica
 Sample 21: setosa
 Sample 22: virginica
 Sample 23: setosa
 Sample 24: virginica
 Sample 25: virginica
 Sample 26: virginica
 Sample 27: virginica
 Sample 28: virginica
 Sample 29: setosa
 Sample 30: setosa

Accuracy: 1.00



```
from google.colab import drive
drive.mount('/content/drive')

import pandas as pd
import numpy as np
from collections import Counter
from sklearn.model_selection import train_test_split

# Load dataset
file_path = "/content/drive/MyDrive/leaf_dataset.csv"
df = pd.read_csv(file_path)

print(df.head())

# CLEAN DATA (VERY IMPORTANT)
df = df.drop(columns=["Unnamed: 5"], errors="ignore") # remove extra column

df = df.dropna() # remove missing rows (fix NaN issue)
```

```

df = df.dropna() # remove missing rows (fix NaN issue)

# Features and label
X = df.drop(columns=["label"]).values.astype(float)
y = df["label"].values

# Train test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Distance function
def euclidean_distance(x1, x2):
    return np.sqrt(np.sum((x1 - x2) ** 2))

# KNN model
class KNN:
    def __init__(self, k=3):
        self.k = k

    def fit(self, X, y):
        self.X_train = X
        self.y_train = y

    def predict(self, X):
        return [self._predict(x) for x in X]

    def _predict(self, x):
        distances = [euclidean_distance(x, x_train) for x_train in self.X_train]

        k_indices = np.argsort(distances)[:self.k]
        k_labels = [self.y_train[i] for i in k_indices]

        return Counter(k_labels).most_common(1)[0][0]

# Train
model = KNN(k=3)
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Accuracy
accuracy = np.mean(y_pred == y_test)

```